

Heart Failure Risk Prediction using MLP Neural Network

This project uses a Multi-Layer Perceptron (MLP) neural network to predict **heart failure survival** outcomes based on clinical records.

The objective is to apply deep learning techniques to healthcare data and evaluate the effectiveness of MLP models for medical risk prediction.

Applications

The objective of this MLP model is to translate static clinical data into actionable insights. It is designed to act as a supportive layer in healthcare systems, assisting professionals in risk assessment and resource management.

Key Objectives:

- **Dynamic Prognostic Simulator:** An interactive system for clinicians to evaluate and project a patient's survival probability across adjustable future time-horizons.
- **Data-Driven Follow-Up Scheduling:** An operational tool for hospitals to optimize post-discharge appointment dates based on personalized high-risk windows.

Dynamic Prognostic Simulator

This application serves as an interactive time-horizon simulator for healthcare professionals.

Upon entering a patient's static clinical biomarkers (such as serum creatinine, ejection fraction, and sodium levels), the physician can dynamically adjust the time feature to simulate different follow-up periods (e.g., 30, 90, or 180 days). The neural network processes these inputs to recalculate and project the survival probability for each specific interval. This dynamic trajectory modeling enables cardiologists to visualize how quickly a patient might approach a critical risk threshold, allowing for highly proactive and personalized intervention strategies.

Dynamic Prognostic Simulator

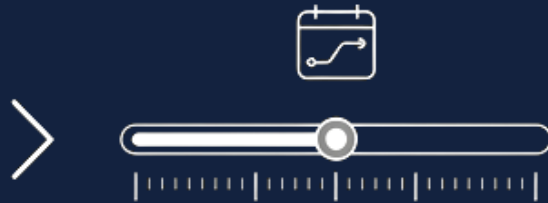
STEP 1

Clinical Data
Collection



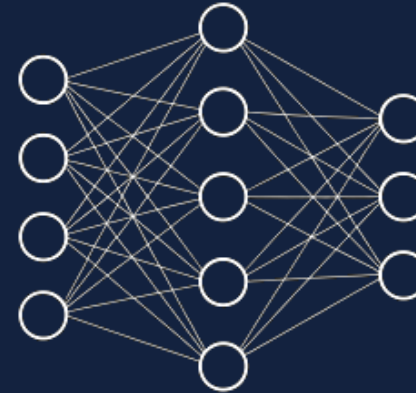
STEP 2

Time Adjustment



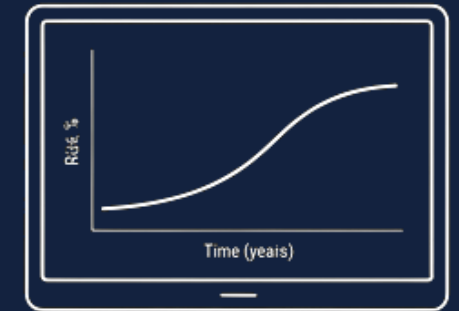
STEP 3

Multi-Layer Perceptron
(MLP)



STEP 4

Patient Future Risk
Probability



Follow-Up Scheduling

A major challenge in healthcare management is determining the optimal time to recall a discharged patient for reassessment.

Hospitals typically rely on fixed, generic schedules (e.g., "return in three months"). This model can serve as the intelligent core of a hospital's scheduling system. By processing the clinical data of a patient, the system predicts the exact timeframe post-discharge when the risk of mortality or heart failure recurrence reaches a critical threshold. Consequently, the hospital can schedule the next follow-up appointment precisely before this high-risk window occurs. This proactive approach not only plays a crucial role in saving lives but also prevents the waste of hospital resources on unnecessary routine check-ups.

Follow-Up Scheduling

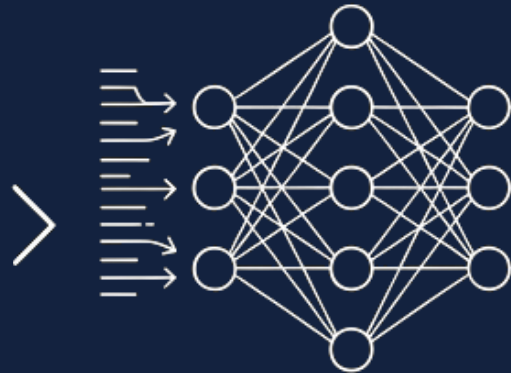
STEP 1

Patient Discharge &
Data Collection



STEP 2

AI Risk Prediction Model



STEP 3

High-Risk Identification



STEP 4

Patient follow-up

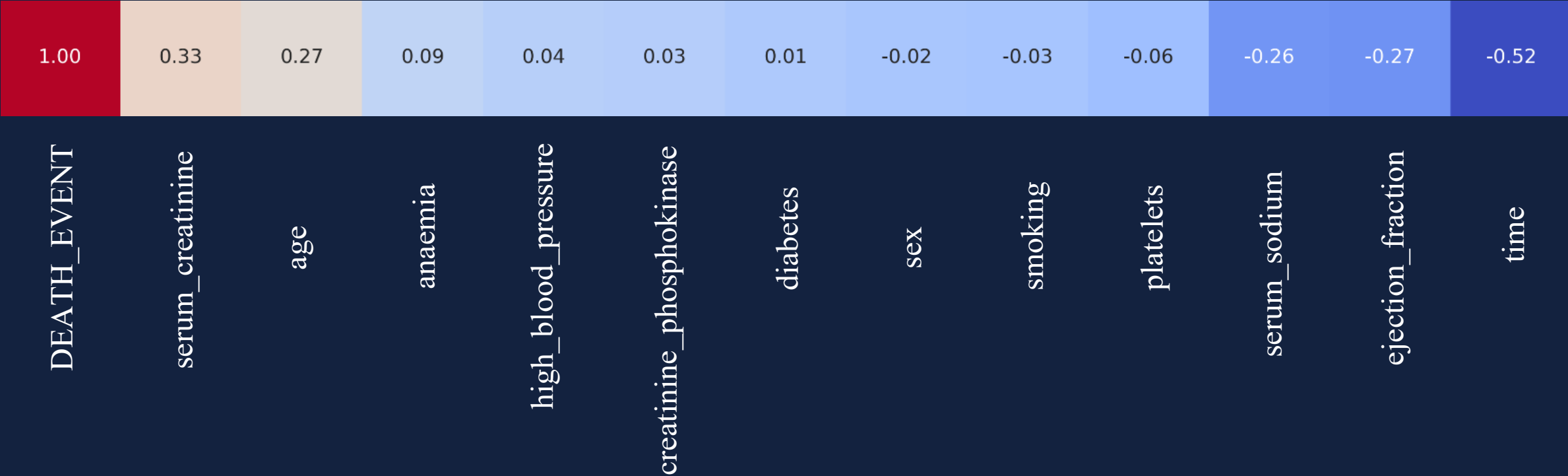


Dataset

age	Age of the patient
anaemia	Decrease of red blood cells or hemoglobin
creatinine_phosphokinase	Level of CPK enzyme in the blood
diabetes	Whether the patient has diabetes
ejection_fraction	Percentage of blood leaving the heart at each contraction
high_blood_pressure	Whether the patient has hypertension
platelets	Platelet count in the blood
serum_creatinine	Level of serum creatinine in the blood
serum_sodium	Level of serum sodium in the blood
sex	Gender of the patient
smoking	Whether the patient smokes
time	Follow-up period (days)
DEATH_EVENT	Whether the patient survived or died

Feature Correlation Analysis

This row of the heatmap shows the correlation of each feature with the target variable (DEATH_EVENT), indicating how strongly and in which direction each feature is linearly related to mortality.



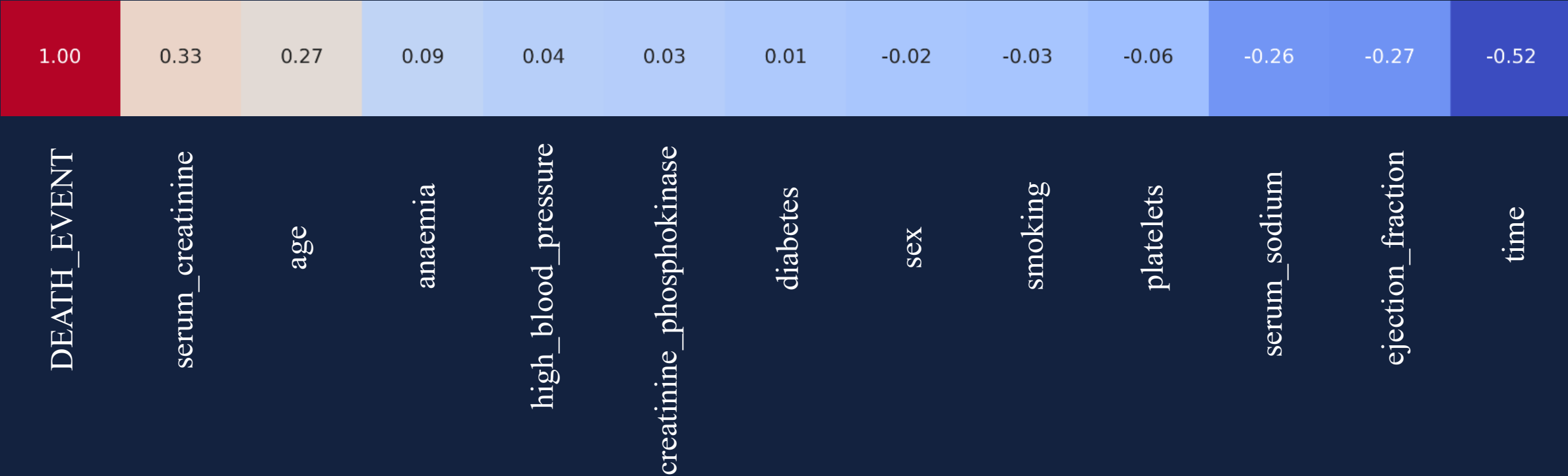
Feature Correlation Analysis

time shows the strongest negative correlation with the target variable.

serum_creatinine and age have moderate positive correlation with mortality.

ejection_fraction and serum_sodium show negative correlation with death risk.

Some features such as diabetes, sex, and smoking have very weak correlation with the target.



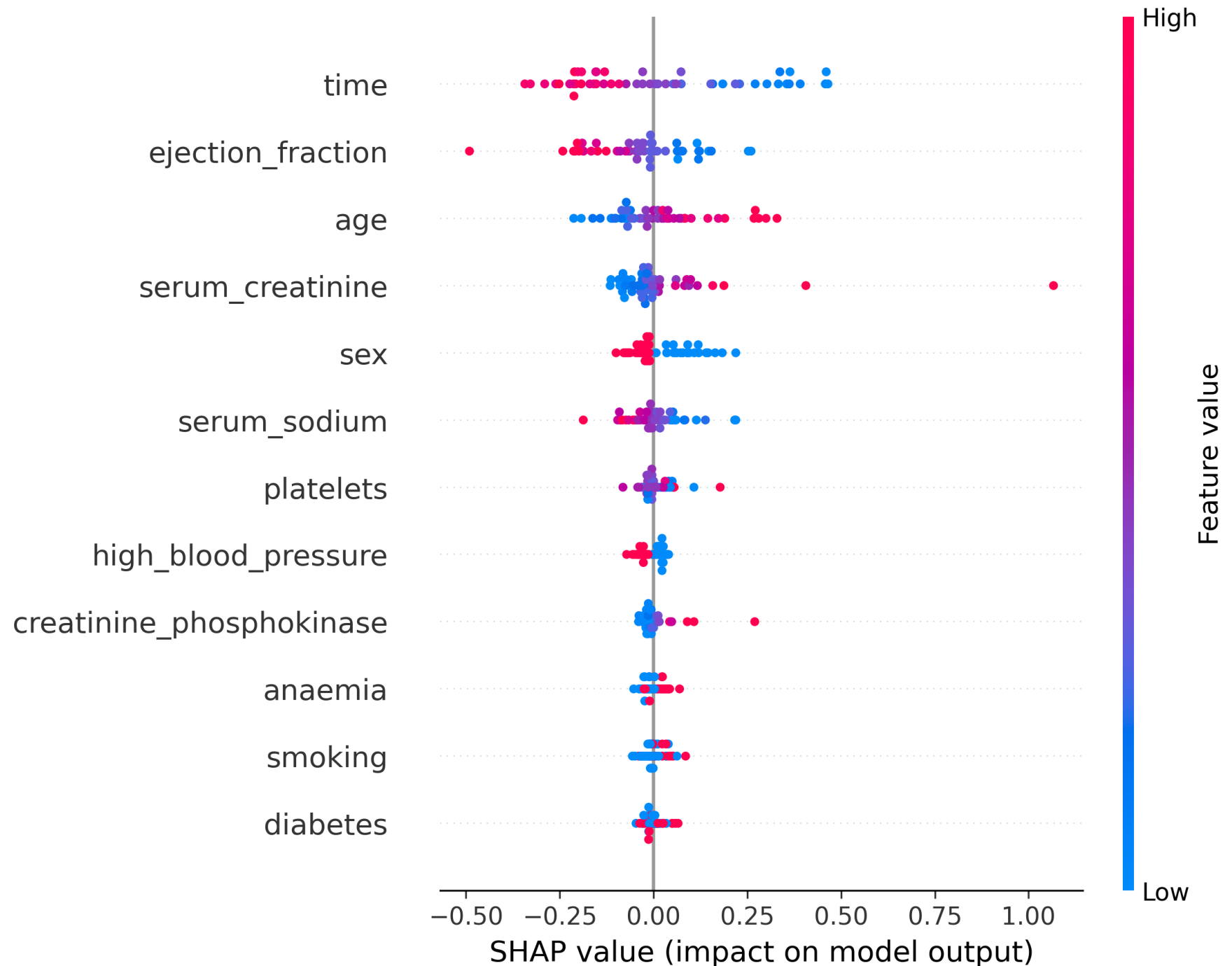
SHAP Analysis

While correlation can highlight some general trends, it cannot capture nonlinear relationships or how features interact in a complex model like an MLP.

To better understand how the model makes predictions, I used SHAP (SHapley Additive exPlanations). SHAP assigns an importance value to each feature for every individual prediction, helping me see how each feature pushes the model's output higher or lower.

Note: This plot does **not** tell which feature is the most important overall. Instead, it shows **how each feature affects the model's decision for individual patients**.

- Each dot in the SHAP summary plot represents a single patient from the test data.
- The position along the x-axis shows whether the feature increases (right) or decreases (left) the predicted risk of death.
- The color of each dot represents the actual value of the feature (e.g., red for high values, blue for low values).
- Features are sorted vertically by their overall impact on model predictions.



Training

Model training was conducted in five different stages. In the first stage, the model was trained using a basic configuration. In stages 2 to 5, different hyperparameters were modified in each experiment to evaluate their impact on the model's behavior, particularly on loss and accuracy trends.

Binary Cross-Entropy (BCE) loss was used in all stages since the problem is a binary classification task involving two classes.

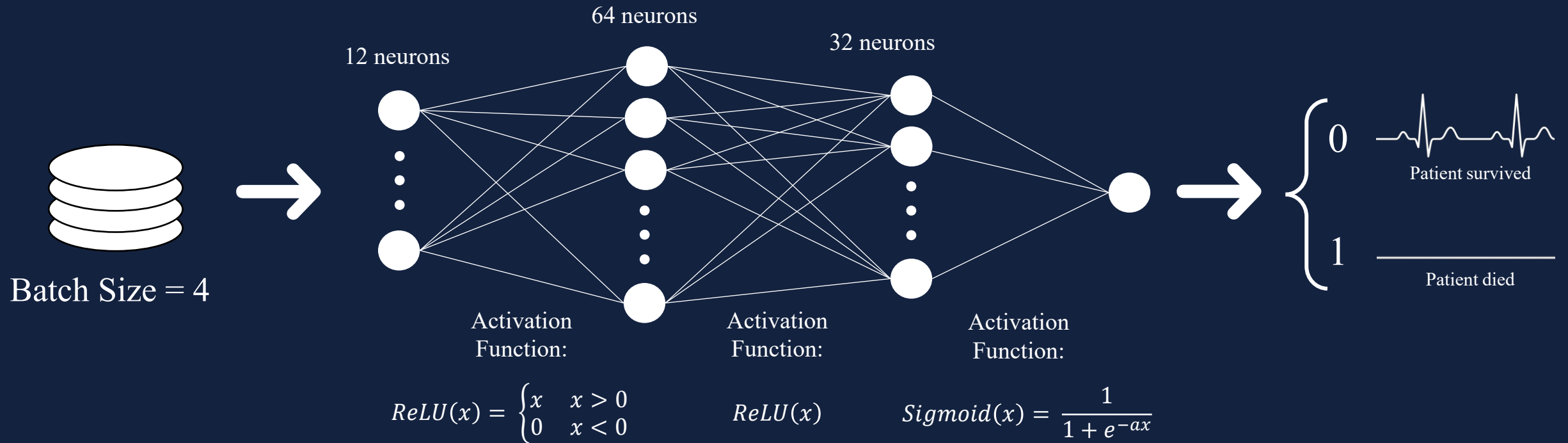
$$L = - \frac{\sum_{i=1}^n (y_{t_i} \log y_{p_i} + (1 - y_{t_i}) \log(1 - y_{p_i}))}{n}$$

The SGD (Stochastic Gradient Descent) optimizer was used throughout all experiments due to its simplicity, stability, and effectiveness in observing how different hyperparameter changes influence the training process.

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} J(\theta)$$

Training – Stage 1

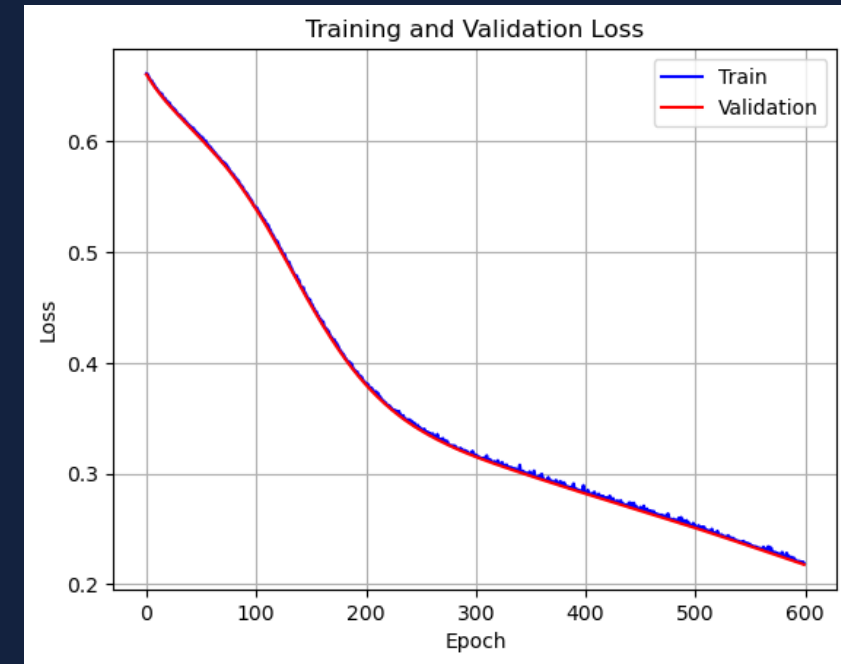
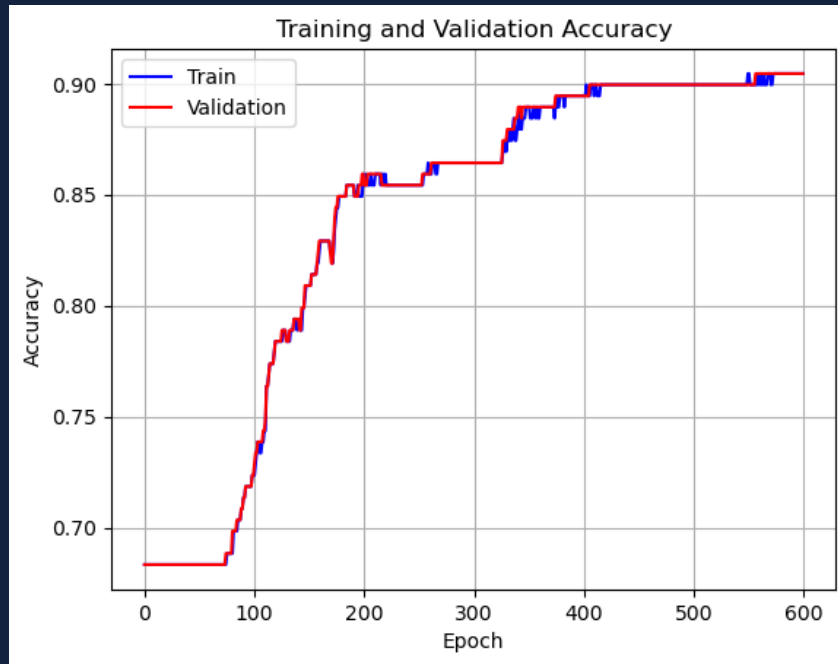
Base Model Training



Learning Rate = 0.001

Training – Stage 1

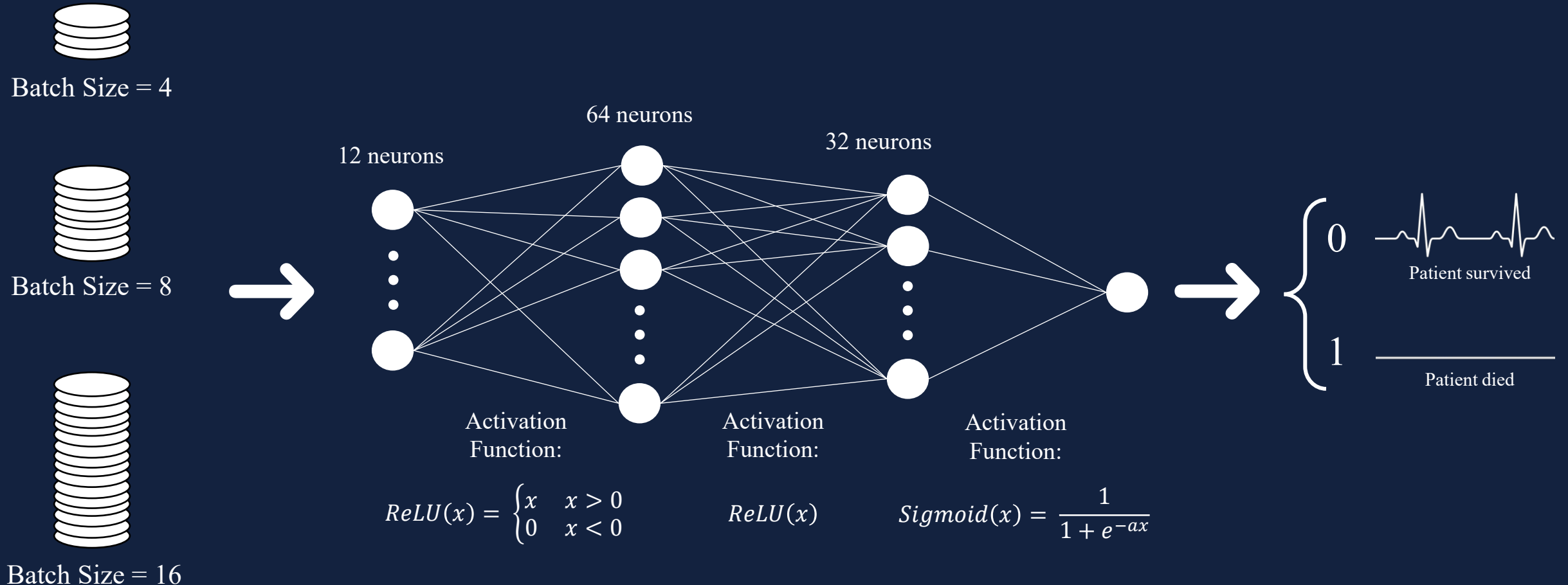
Results Analysis



Although the model achieved extremely high accuracy and very low loss on both the training and validation sets, this behavior is not necessarily desirable because it indicates possible overfitting. Due to the small dataset size, the model likely learned dataset-specific patterns instead of generalizable features. As a result, despite the near-perfect training and validation performance, the model achieved only 69.39% accuracy on the test set, showing weak generalization to unseen data.

Training – Stage 2

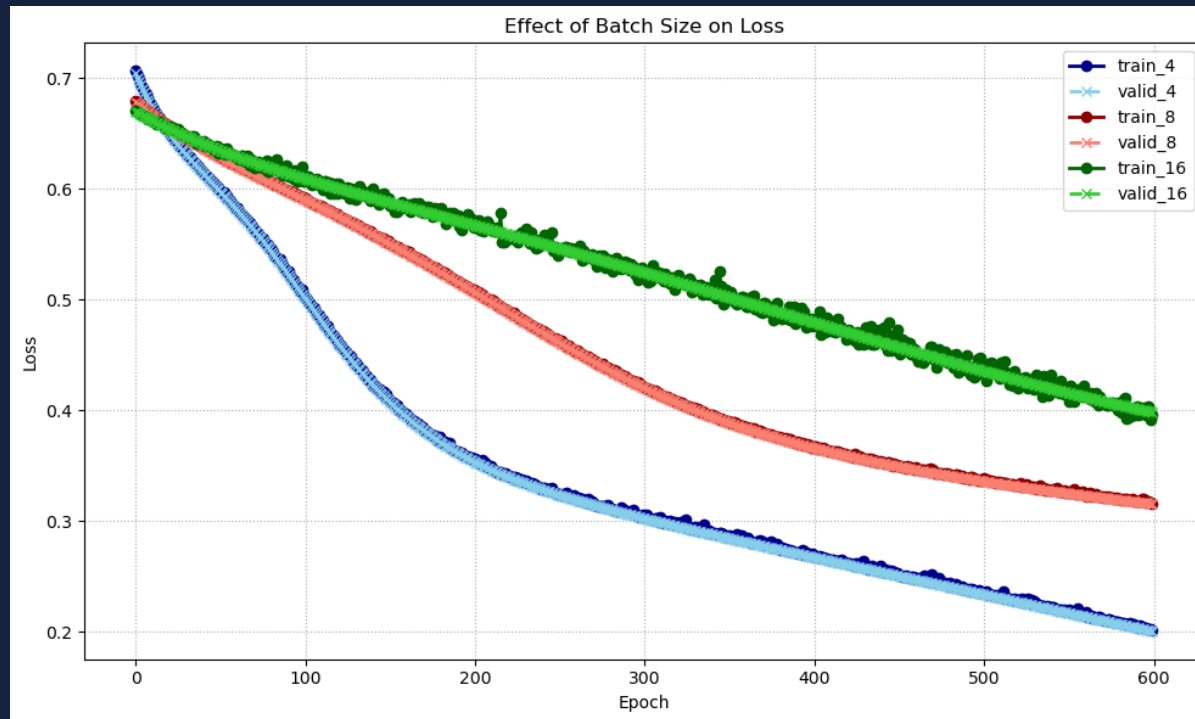
Compare different batch sizes and loss behavior



Learning Rate = 0.001

Training – Stage 2

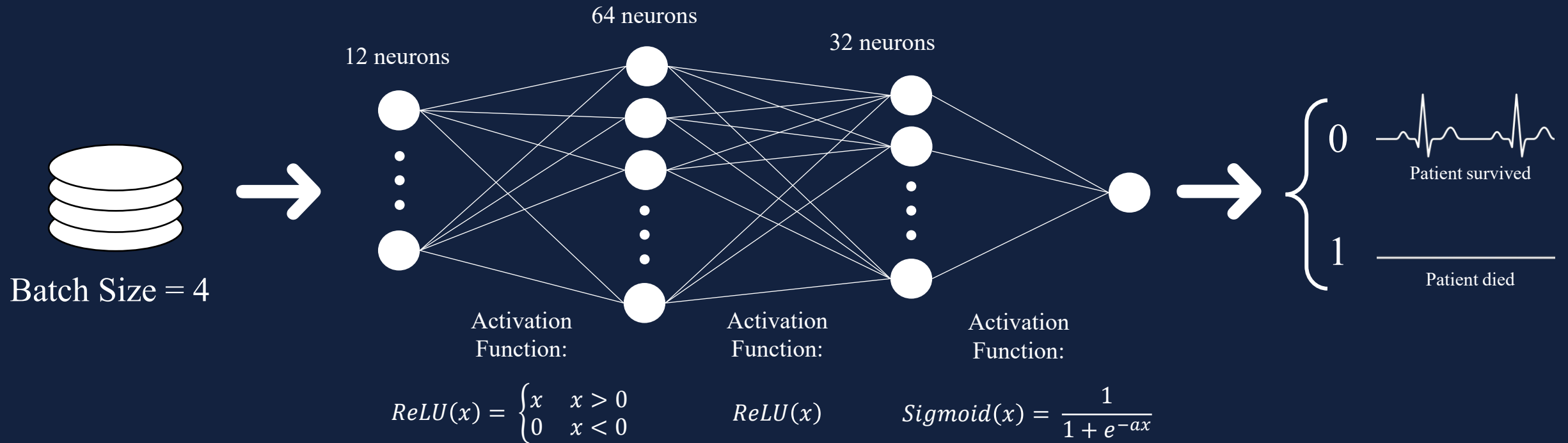
Results Analysis



Best Model Performance: Batch Size 4 > Batch Size 8 > Batch Size 16

Training – Stage 3

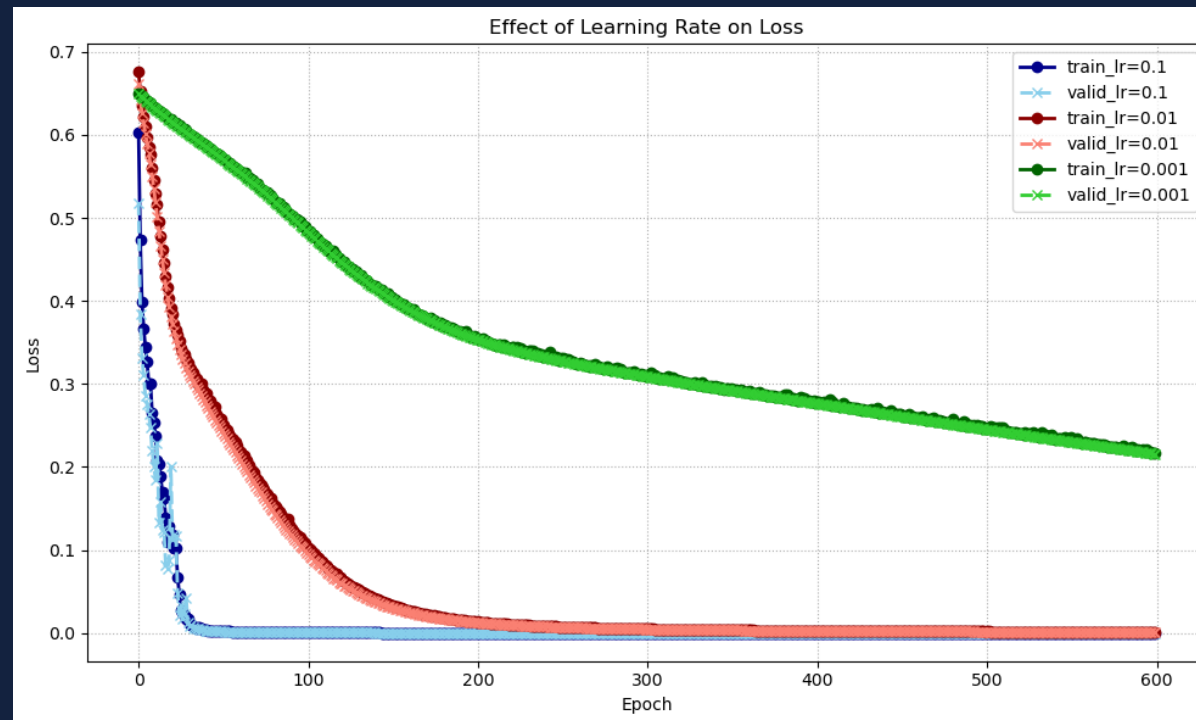
Compare different learning rates and loss behavior



Learning Rate = 0.1, 0.01, 0.001

Training – Stage 3

Results Analysis

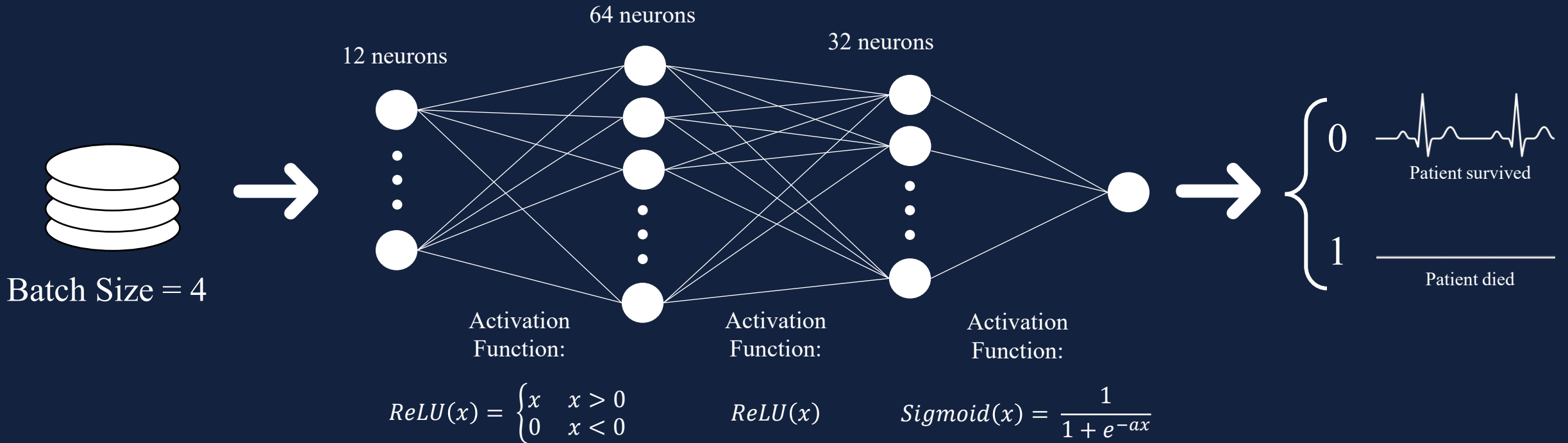


Best Model Performance: Learning Rate 0.1 > Learning Rate 0.01 > Learning Rate 0.001

Training – Stage 4

Compare different number of hidden layers and loss behavior

Hidden Layers = 2

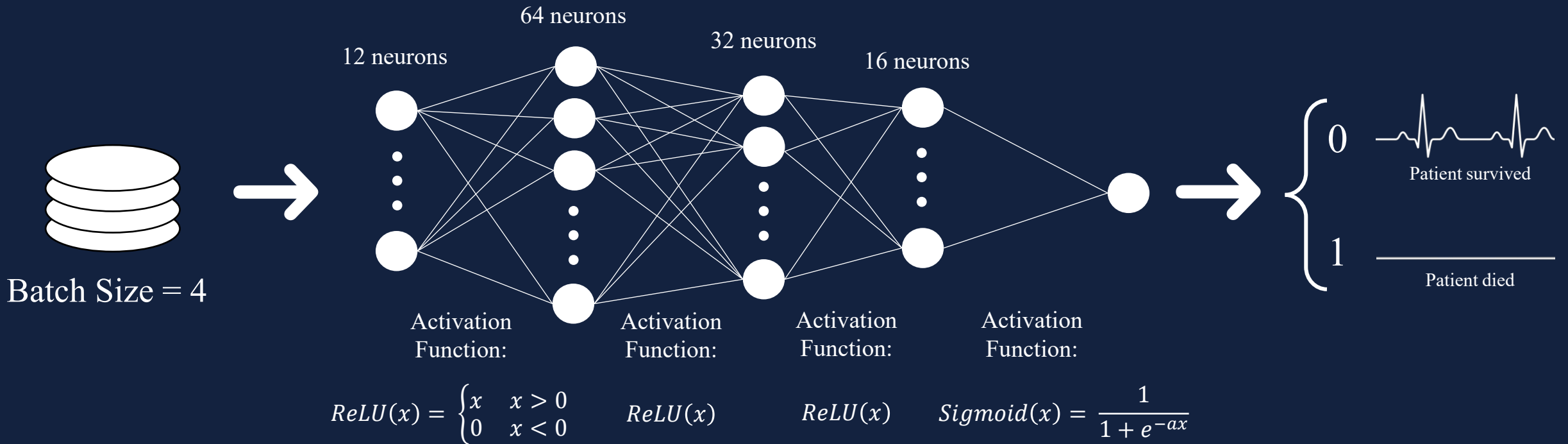


Learning Rate = 0.001

Training – Stage 4

Compare different number of hidden layers and loss behavior

Hidden Layers = 3

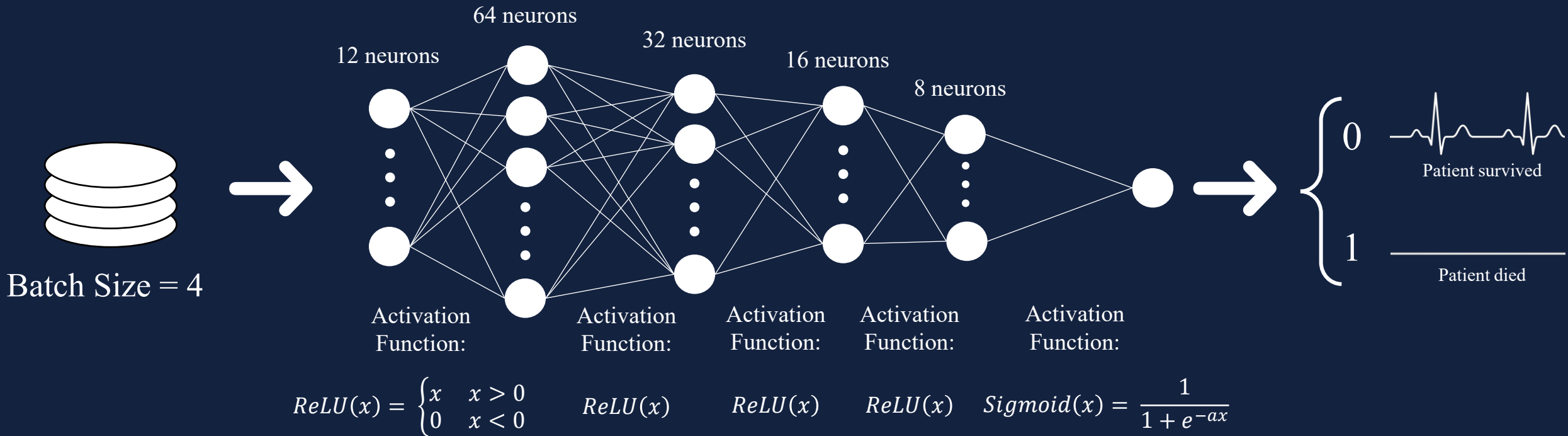


Learning Rate = 0.001

Training – Stage 4

Compare different number of hidden layers and loss behavior

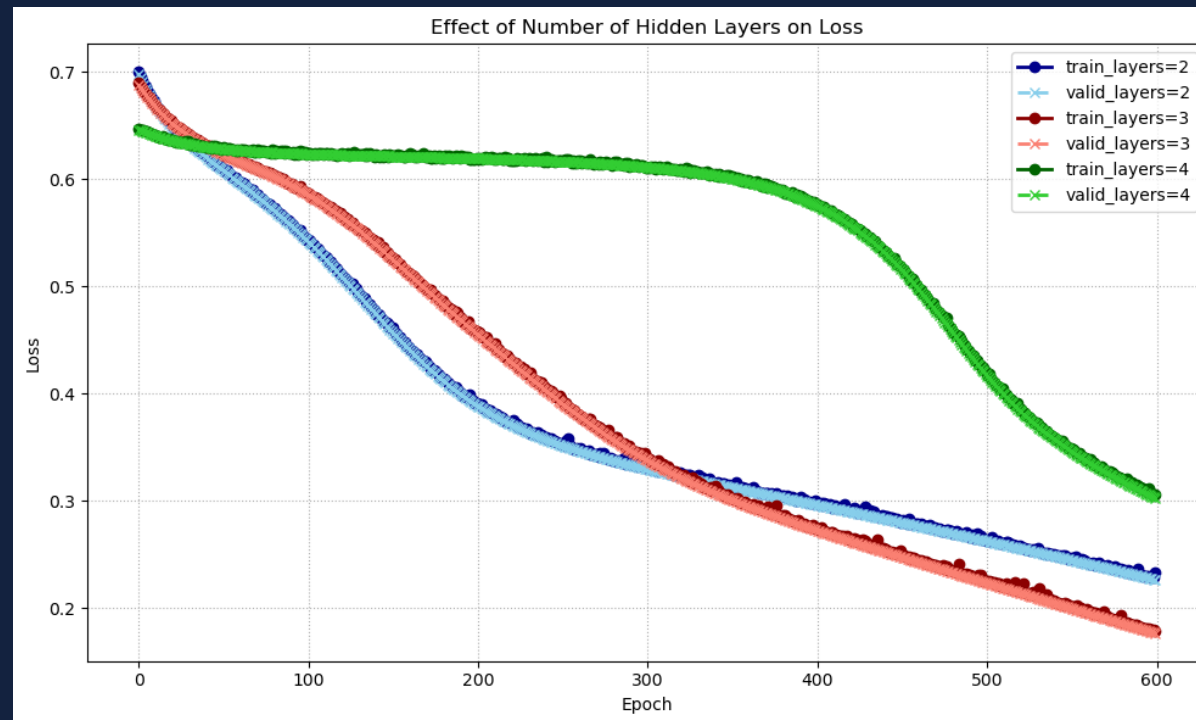
Hidden Layers = 4



Learning Rate = 0.001

Training – Stage 4

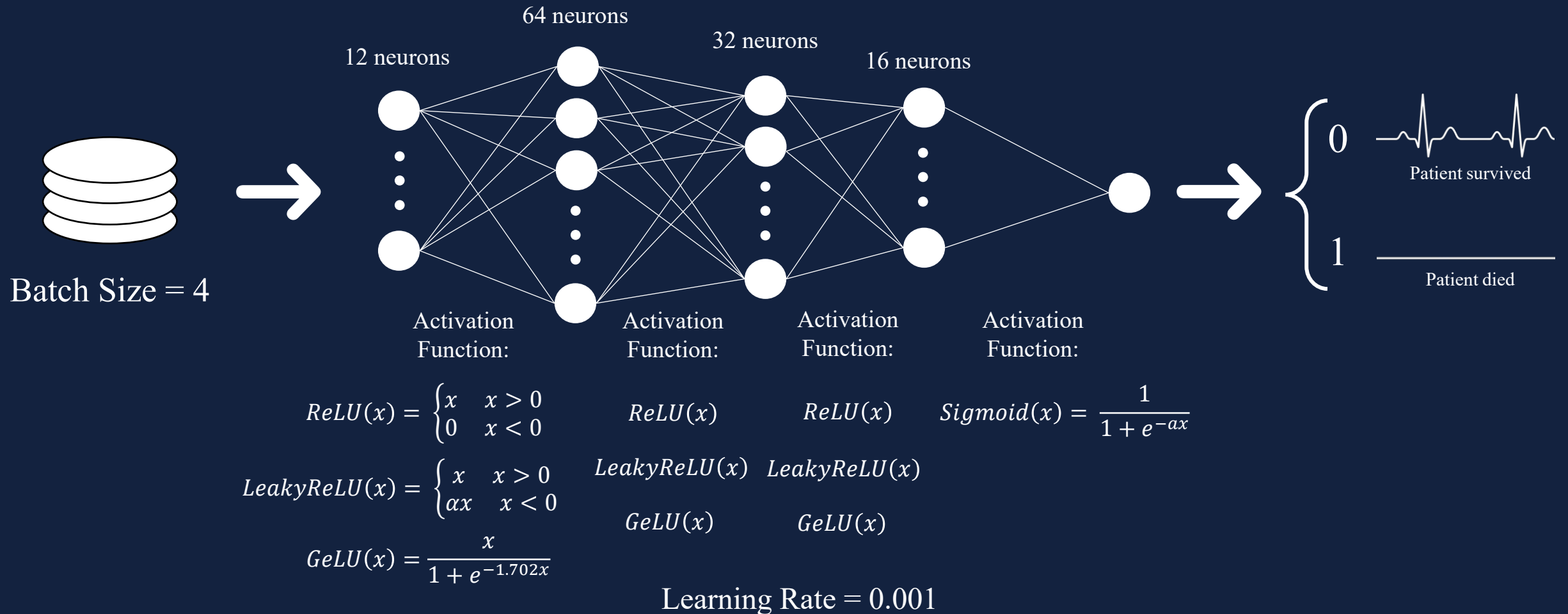
Results Analysis



Best Model Performance: 3 Layers > 2 Layers > 4 Layers

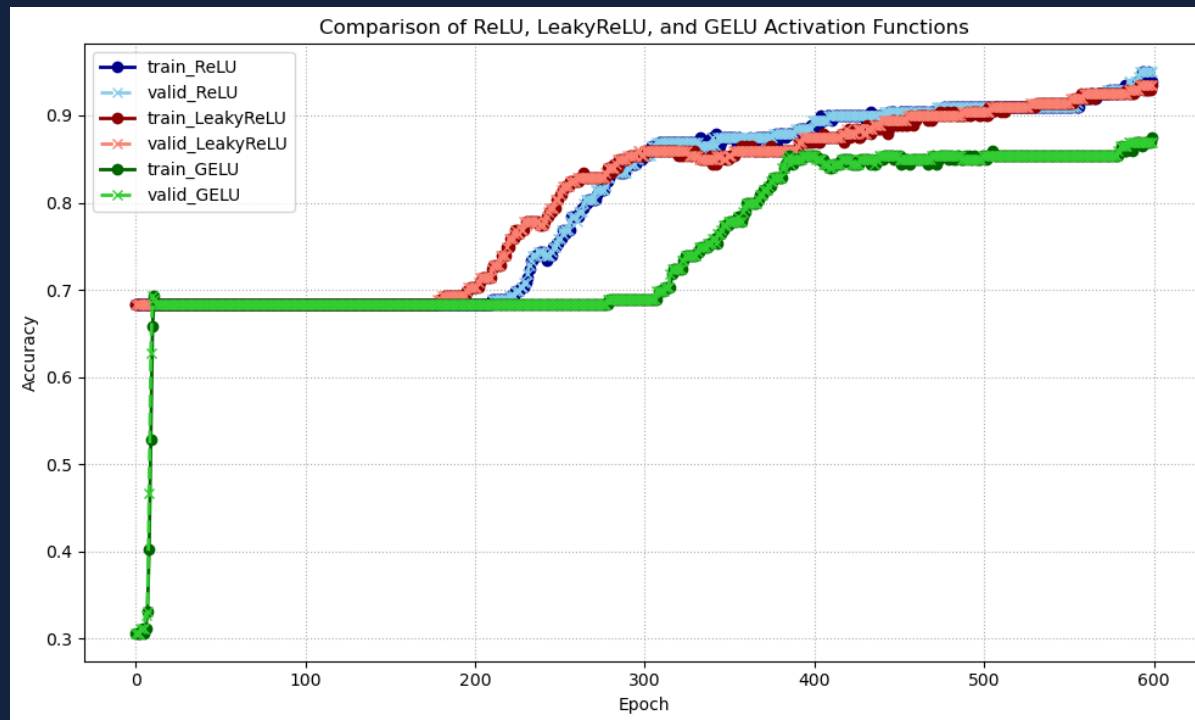
Training – Stage 5

Compare different activation functions and accuracy behavior



Training – Stage 5

Results Analysis



Best Model Performance: $\text{ReLU} > \text{LeakyReLU} > \text{GeLU}$



Daniyal Iran Mehr

Artificial Intelligence | Data Science

Hi there! I'm a computer engineer passionate about Artificial Intelligence and Data Science. I truly believe this field is shaping our future, and I'm on a journey to continuously learn and make a meaningful impact. Feel free to follow my work if you're interested. I'm always excited to connect and collaborate with like-minded people in this space.



github.com/Daniyaliranmehr



linkedin.com/in/daniyaliranmehr



daniyaliranmehr@gmail.com